

武汉科技大学

本科毕业论文

题目：基于 Java 的体育馆管理系统设计
与实现

学 院： 计算机科学与技术学院

专 业： 网络工程（国际）

学 号： 202221190089

学生姓名： 谭雨基

指导教师： 导师姓名 (职称)

日 期： 2026 年 5 月 6 日

摘 要

高校体育馆的日常运营涉及场地排期、器材出借和订单结算等多类业务，多数学校长期沿用人工登记加电话沟通的方式处理这些事务。这种方式在学生使用频次较高的时段尤其容易出现重复预约、信息更新不及时和事后对账困难等情况。本文针对上述问题，开发了一套面向校园场景的体育馆管理系统，希望把分散的登记、支付和租借动作收拢到一个平台里。

系统整体采用前后端分离实现。前端用 Vue3 搭配 Element Plus 完成用户端和管理端两套界面，所有接口请求由 Axios 统一封装；后端使用 Spring Boot 提供 RESTful 接口，数据访问层基于 MyBatis-Plus，数据库选用 MySQL；登录后由 JWT 携带身份信息进出，权限判断放在拦截器里完成。功能上，用户可以浏览场地、按日期挑选时段下单、随后支付或取消订单，也可以提交器材租借申请并查看自己的租借记录；管理员则负责场地、订单、用户和器材这四类数据的维护。

实现过程中，预约时段冲突判断、订单状态机、器材库存扣减与归还、以及后台首页的数据汇总属于较关键的几处逻辑，本文也对这几个点的处理方式做了较为详细的说明。系统在本地与测试环境完成了功能验证，常规业务路径均能按预期运行，可作为同类校园资源管理系统的参考实现。

关键词：体育馆管理系统；场地预约；Spring Boot；Vue3；JWT

Abstract

With the continuous development of campus sports activities, the traditional management mode of gymnasiums based on manual registration and offline communication can no longer satisfy the needs of venue reservation, equipment rental and daily operation management. To solve the problems of booking conflicts, delayed information updates, low statistical efficiency and poor user experience, this thesis designs and implements a gymnasium management system based on Java to improve the efficiency of resource allocation and digital management in campus gymnasiums.

The system adopts a front-end and back-end separated architecture. Vue3 and Element Plus are used to build both user-side and administrator-side interfaces, while Spring Boot, MyBatis-Plus and MySQL are employed to implement business logic and data persistence. JWT is introduced to support identity authentication and permission control. The main functions of the system include user registration and login, venue information query, online reservation, order payment and cancellation, equipment rental and record query, as well as unified management of venues, orders, users and equipment by administrators.

This thesis carries out the research from the aspects of requirement analysis, overall design, database design, function implementation and system testing. Special attention is paid to the implementation of booking conflict detection, order status transition, inventory maintenance and dashboard statistics. The test results show that the system can stably support the daily business of campus gymnasiums, and it has good practicability and scalability for further application and improvement.

Key words: Gymnasium management system; venue reservation; Spring Boot; Vue3; JWT

目录

摘要.....	I
Abstract.....	II
第 1 章 绪论.....	1
1.1 研究背景与意义.....	1
1.1.1 研究背景.....	1
1.1.2 研究意义.....	1
1.2 国内外研究现状.....	1
1.2.1 国外研究现状.....	1
1.2.2 国内研究现状.....	2
1.3 研究内容与论文结构.....	2
1.3.1 研究内容.....	2
1.3.2 论文结构安排.....	3
第 2 章 相关技术与理论基础.....	4
2.1 前端开发技术.....	4
2.1.1 Vue3.....	4
2.1.2 Element Plus.....	4
2.2 后端开发技术.....	4
2.2.1 Spring Boot.....	4
2.2.2 JWT 认证机制与接口设计.....	5
2.3 数据存储与开发环境.....	5
2.3.1 MySQL 数据库技术.....	5
2.3.2 系统开发与运行环境.....	5
第 3 章 系统需求分析.....	6
3.1 可行性分析.....	6
3.1.1 技术可行性分析.....	6
3.1.2 经济与操作可行性分析.....	6
3.2 功能需求分析.....	6
3.2.1 用户端功能需求分析.....	6
3.2.2 管理端功能需求分析.....	7
3.3 非功能需求分析.....	8
3.3.1 性能与安全需求分析.....	8
3.3.2 易用性与可维护性分析.....	8

第 4 章 系统总体设计	10
4.1 系统架构设计	10
4.1.1 前后端分离架构设计	10
4.1.2 系统模块划分设计	10
4.1.3 核心业务流程设计	11
4.2 数据库设计	13
4.2.1 数据库概念结构设计	13
4.2.2 数据库逻辑结构设计	13
第 5 章 系统详细设计与实现	16
5.1 场地列表与筛选实现	16
5.2 预约时段选择与下单实现	16
5.3 订单列表与状态操作实现	17
5.4 器材租借与记录查询实现	18
5.5 场地管理页面实现	18
5.6 订单与用户管理实现	19
5.7 器材信息维护实现	20
5.8 数据统计看板实现	20
第 6 章 系统测试	22
6.1 测试环境与方法	22
6.2 功能测试分析	22
第 7 章 总结与展望	24
7.1 研究总结	24
7.2 后续展望	24
参考文献	26
致谢	27

图目录

图 3.1 用户端功能用例图	7
图 3.2 管理端功能用例图	8
图 4.1 前后端分离架构图	10
图 4.2 系统功能模块图	11
图 4.3 核心业务流程图	12
图 4.4 数据库 ER 图	13
图 5.1 场地列表与筛选页面	16
图 5.2 预约时段选择与下单页面	17
图 5.3 订单列表与状态操作页面	17
图 5.4 器材租借与记录查询页面	18
图 5.5 场地管理页面	19
图 5.6 订单管理页面	19
图 5.7 用户管理页面	20
图 5.8 器材信息维护页面	20
图 5.9 数据统计看板页面	21

表目录

表 4.1 角色表结构设计	14
表 4.2 用户表结构设计	14
表 4.3 场地表结构设计	14
表 4.4 订单表结构设计	14
表 4.5 器材表结构设计	15
表 4.6 租借记录表结构设计	15
表 6.1 系统测试环境	22
表 6.2 核心功能测试结果	23

第 1 章 绪论

1.1 研究背景与意义

1.1.1 研究背景

近几年来，高校信息化建设速度加快，以前依靠人工记录、打电话排班等手段进行事务管理的事务渐渐迁移到了线上，其中也包含体育馆的日常管理工作。对于高校体育馆这个场景来说，处理起来并不难，只是排场地、收钱、借还器材、统计运营数据而已，不过一遇一起，这些事情就显得零散了——信息只在管理员手里，学生看不到实时状态；同一时间段内有两人的预约出现；月末统计需要翻台账。此类问题需要依靠人工来维持已经变得越来越困难，并且无法给师生提供一个稳定的预约环境。

从使用方的角度来说，普通用户希望随时随地可以打开手机或者电脑查询当前有哪些场地空闲、可以直接选择时段下单、可以看到自己的历史订单；管理员更加关心目前场地是否有人正在使用、器材还有多少、最近的订单和注册情况是怎样的。将这两种需求放在同一个系统中，就是本课题的出发点。

1.1.2 研究意义

本课题的实际意义主要体现在两个比较具体的地方。第一种就是减少体育馆日常管理中重复登记的工作量。以预约为例，如果仍然依靠管理员人工记录，用户每次咨询都要先确认场地、日期和时段，管理员再去核对已有的记录；本系统把场地状态、预约时段、订单状态都存入数据库，用户在页面上即可进行查询与下单，管理员只需处理场地上下架、异常订单、器材归还等少数几项工作。第二是保留业务数据。订单、租借记录、用户、场地信息等都存于表结构当中，以后不管是看高峰时段还是统计器材使用情况，都会比纸质台账容易整理。

从毕业设计训练的角度来说，本系统不是单纯的完成几个页面，而是把一个小型 Web 系统从需求到实现串联起来。开发中会遇到前端路由和页面状态，后端接口分层，数据库表关系，JWT 登录鉴权，订单状态流转和器材库存扣减等问题。所有的内容都和真实项目的基础工程能力相关。系统规模小、没有接入真实的支付、校园统一认证，但是它已经包含了校园资源预约系统中典型的一组业务流程，因此对于类似的场地、教室或者器材管理系统的开发也有一定的参考意义。

1.2 国内外研究现状

1.2.1 国外研究现状

在国外大学、公共体育场馆早就建有线上预约系统。早期系统大多与学校统一身份认证、校园卡或者会员账户绑定在一起，主要解决用户的身份识别、场地的预约、费用的结算以及入馆的核验等问题^[1]。平台使用时间越久，系统里面留存的订单以及访问数

据也就越来越多，有关的研究也开始重视数据分析方面的技能，比如依照历史预约记录来推测热门时段，依照项目偏好给出场地的推荐，或者借助统计的结果来协助场馆安排开放时间^[2]。

国外的相关系统发展大体上是由“线上化预约”到“平台化运营”。它们一般不会只强调某个功能，而更加重视账号体系、支付系统、门禁设备以及数据分析之间的相互配合。但是此类方案的建设成本及维护费用较高，不适合直接套用到本科毕设规模的校园体育馆系统之中。对于本课题而言，更具有参考意义的是它的模块化思想，即预约、订单、用户、权限、统计之间不能混在一起，而应该保持一定的边界。

1.2.2 国内研究现状

国内有关校园公共资源管理系统的相关研究虽然数量不少，但研究对象主要是教室、实验室、图书馆座位、体育场馆等。技术实现上使用的是 Spring Boot、Vue、MySQL 的组合，在论文中会以登录注册、资源查询、预约申请、后台维护、基础统计等为主要功能^[3]。这类系统的开发难度小、数据完整度高，比较适合用作高校毕业设计或者中小型管理系统的技术方案^[4]。

但已有研究中也存在着一些容易被忽视的问题。部分系统将重点放在页面是否完整上，而缺少对异常路径的说明，比如重复预约怎样判断、订单取消后状态怎样变化、普通用户能否访问管理接口等等；有些系统的数据统计只是简单地显示数量，并没有和实际的管理决策紧密相连^[5]。本课题在实现过程中也受到了以上问题的提醒，在需求分析和设计章节中会单独说明预约冲突校验、角色权限控制、库存扣减和后台数据概览等内容。这样写的不是为了堆叠功能，而是使系统的主体业务流程可以闭环。

1.3 研究内容与论文结构

1.3.1 研究内容

本文主要对体育馆管理系统的开发与实现进行研究，分为三部分。第一层为业务流程梳理，主要是对普通用户和管理员两种角色的操作进行分析，用户侧包含注册登录、浏览场地、选择日期和时段、提交预约、支付或者取消订单、租借器材、查看个人记录；管理侧包含维护场地信息、查看订单、管理用户角色、调整器材库存、处理归还、查看后台概览数据。

第二层就是系统的设计。本文采用前后端分离结构，前端使用 Vue3 和 Element Plus 完成页面与交互，后端使用 Spring Boot、MyBatis-Plus 和 MySQL 完成接口、业务逻辑和数据持久化。数据库的设计以角色、用户、场地、订单、器材、租借记录这六种实体为依托，用状态字段来表现订单支付、取消以及器材归还等业务的变化。

第三层就是功能实现和测试。系统实现了场地分页查询、预约时段展示、订单生成和状态操作、器材租借和归还、后台管理以及数据看板等模块的功能，使用登录认证、预约冲突、库存不足、权限访问等测试用例对主要流程进行了测试。由于本课题时间和

场景限制，本系统目前没有接入真实支付、短信提醒、校园统一身份认证等模块，将在以后的展望中予以说明。

1.3.2 论文结构安排

本文一共分为七章。第 1 章为绪论，主要对课题研究背景、研究意义、国内外研究现状、本文的研究内容和论文结构进行介绍；第 2 章为相关的技术、理论基础，介绍系统实现中使用到的各种技术、工具；第 3 章为系统的需求分析，从可行性、功能需求、非功能需求等几个方面说明系统建设的依据。

第四章系统总体设计，主要是对系统架构设计、数据库设计、关键设计方案的论述；第五章系统详细设计与实现，主要是说明各个功能模块的页面实现和业务实现过程；第六章系统测试，对系统运行环境、测试方法、测试结果进行分析；第七章总结与展望，对全文工作进行归纳总结，并提出系统后续完善的方向。

第 2 章 相关技术与理论基础

2.1 前端开发技术

2.1.1 Vue3

本系统前端使用 Vue3。选择它并不是因为它是主流，而是因为项目页面之间存在着比较明显的状态差别，用户端有登录、场地列表、预约、器材租借、个人订单等页面，管理端有场地维护、订单管理、用户管理、数据看板。直接用 HTML 和零星的脚本来实现页面切换、表单回显、分页刷新等逻辑，会使页面的组织变得凌乱。Vue3 的组件化方式可以把每一个页面拆分成独立的视图，接口请求则集中在 api 目录下，代码结构也会更加清晰^[6]。

Vue3 的响应式机制主要是用在几类操作上。预约页面根据场地开放时间产生可选时段，用户切换日期之后再次请求 `/api/venues/{id}/schedule`，页面会自动更新已约时段和可选结束时间；订单页在支付或者取消之后重新加载列表，状态标签随着返回数据的变化；后台看板使用 `computed` 统计开放场地数、已支付订单数、器材可借率。这些地方若用手工操作 DOM，则代码会变得繁杂，Vue3 可以将数据的变化同界面的更新绑定在一起，适合于本系统这类交互并不复杂但是状态较多的业务页面。

2.1.2 Element Plus

Element Plus 是给 Vue3 用的一套较常见的组件库。本项目中能看见它的地方主要是表格、表单、弹窗、分页、消息提示这五类——后台页面全部使用了 `el-table` 和 `el-dialog`，前台的下单、租借也是用 `el-form` 配合校验规则来实现数据的收集；操作反馈全部通过 `ElMessage` 和 `ElMessageBox` 来完成^[7]。这些控件的样式以及交互行为较为成熟，可以省去自己写组件、调样式的时间，对于一个学生级的项目来说特别合适。

风格上的好处是统一性。整个系统从前台到后台使用相同的配色、控件，按钮、表格、提示弹层都不会有前后页面不同造成的违和感，上手更简单。

2.2 后端开发技术

2.2.1 Spring Boot

后端采用 Spring Boot 2.7。它之所以被选中是因为 starter 依赖已经将 `web`、`validation`、`jdbc` 等常用东西集成了起来，消除了 Spring 早期版本大段 XML 配置的需求，项目按 `controller`、`service`、`mapper` 三层分开了来，每个模块的登录、场地、订单、器材等模块都在一个包内，新人接下来看代码可以快速找到对应部分^[8]。

在实现上，Spring Boot 只做“基建”，即接口暴露、参数绑定、参数校验和全局异常处理等工作。看起来很简单的问题，却需要在每个控制器里重新写相同的参数判空和统一返回包装代码，整体可读性就会下降很多。MyBatis-Plus、JWT 工具类、数据库连

接池等组件使用 Spring 的依赖注入方式一起工作，实际开发业务的时候只需要关注自己的需求就可以了。

2.2.2 JWT 认证机制与接口设计

登录鉴权选用了 JWT。它的特点就是无状态，服务端不需要保存会话，token 本身就包含有用户 ID 和角色信息，后端拦截器拿到 token 解析之后就可以直接判断了。这种结构在前后端分离的模式下比较方便，也不必在多实例环境下还要进行 session 共享^[9]。本系统的做法是登录成功后把 token 通过响应头返回前端，后续请求由 Axios 拦截器统一塞进 Authorization 头，再由后端 JwtInterceptor 解析。

接口设计上遵循 RESTful 风格，路径按认证、场地、订单和管理端等业务域划分，所有响应都用一个统一的 Result<T> 对象包装，里面是 code、message、data 三个字段。这样的好处是前端处理成功和失败可以走同一条分支：先看 code，再决定怎么用 data。

2.3 数据存储与开发环境

2.3.1 MySQL 数据库技术

数据存储用的是 MySQL 8.0。对于一个校园规模的系统来说，关系型数据库可以满足基本的要求，社区资料也比较齐全，调试、排错都很方便。本系统的数据模型是以角色、用户、场地、订单、器材、租借记录六个表为基础构建的，外键以及状态字段把这些实体联系在一起，从而形成完整的业务语义^[10]。

数据库不仅仅负责信息的存储，预约冲突判断、订单状态转换、库存扣减等逻辑也被直接放在了 SQL 里实现，冲突判断用的是按照 venue_id 和 book_date 进行范围查询再加唯一索引，库存扣减则放进了 @Transactional 中防止并发导致的数量出现负数。这些决定着业务能否正常运转。

2.3.2 系统开发与运行环境

开发环境的具体版本是 Java 17、Spring Boot 2.7、MyBatis-Plus 3.5、MySQL 8.0、Vue3 + Vite、Node.js 18。后端使用 Maven 管理依赖，前端使用 Vite 启动开发服务，并进行热更新以及构建。组合是目前 Web 项目中比较常见的配置，社区活跃，遇到问题很容易找到现成的方案。

部署时前后端可以分别独立运行，前端构建产物是一组静态文件，用 Nginx 或任意静态托管即可，后端打包成 jar 文件直接 java -jar 启动，数据库独立部署。这样一种结构使得三者可以分阶段地添加或者缩减，当压力过大时可以单独增加一台后端的机器而不需要动用全部系统的资源。

第 3 章 系统需求分析

3.1 可行性分析

3.1.1 技术可行性分析

从技术条件看，本课题采用的方案是可完成的。前端 Vue3 负责页面和路由，后端 Spring Boot 提供 REST 接口，MyBatis-Plus 完成基础增删改查，MySQL 保存业务数据，JWT 用于登录后的身份传递。这些技术在课程学习以及日常开发的资料当中比较常见，在出现问题的时候能从官方文档、示例项目、调试日志中定位。对于本科毕设来说，技术栈中并没有加入过于复杂的分布式组件，也没有复杂的部署环境，因此开发和演示成本可控。

本系统的主要业务问题就是两个。第一就是预约冲突判断，后端根据场地编号、预约日期和起止时间来查询是否存在重叠订单，并且已经支付和未支付的订单都应该占用时间段；第二就是权限隔离，普通用户只能看到自己的订单和租借记录，管理员才能看到后台接口。以上问题都可以在现有的框架里解决，将预约流程放到事务方法里完成，把权限信息通过 JWT 解析后存入当前请求上下文中，在管理接口调用之前再做角色判断。因此从技术实现角度来讲，该课题是可行的。

3.1.2 经济与操作可行性分析

经济层面的成本很小。整套系统全部依靠开源生态系统，Java、Spring Boot、Vue 等都没有需要付费的许可，对硬件要求也很低，普通的服务器就足够了，对一所高校来说部署成本可以忽略不计。开发上线之后，原来的需要管理员人工处理的预约登记、订单核对等操作都可以被系统自动完成，节省的人力可以弥补前期的投资。

采用两种不同的角色。普通学生用户的操作路径非常简单，只需要登录、找场地、订时段、付款或者取消，平时使用过一次以上的 App 就基本可以操作；管理员的页面虽然很多，但是都是常见的“列表+表单+弹窗”，对于经常接触办公系统的人来说不需要再做培训。综上所述，系统对于两类使用者来说都没有存在适应门槛。

3.2 功能需求分析

3.2.1 用户端功能需求分析

用户端主要针对到馆使用师生。最基础的入口就是注册、登录，进入系统后用户会做以下两项基本操作，即场地预约、器材租借。前者从浏览场地、按类型筛选、选择日期看可用时段开始，提交预约、生成订单，事后支付或者取消；后者则是在个人页面里查看可借器材、提交租借申请，在个人页面上查看历史记录。两类业务包括普通用户在体育馆里能够产生的大部分动作。

设计时我们关注的不是功能是否齐全，而是每一个步骤下用户都能看到自己想要的信息。选时段时，已占部分应该明示标出，用户提交之后才会发现冲突；订单页应该明确标出当前是“待支付”还是“已完成”，对应的可以进行的操作有哪些。这些就构成了一个系统的优劣。

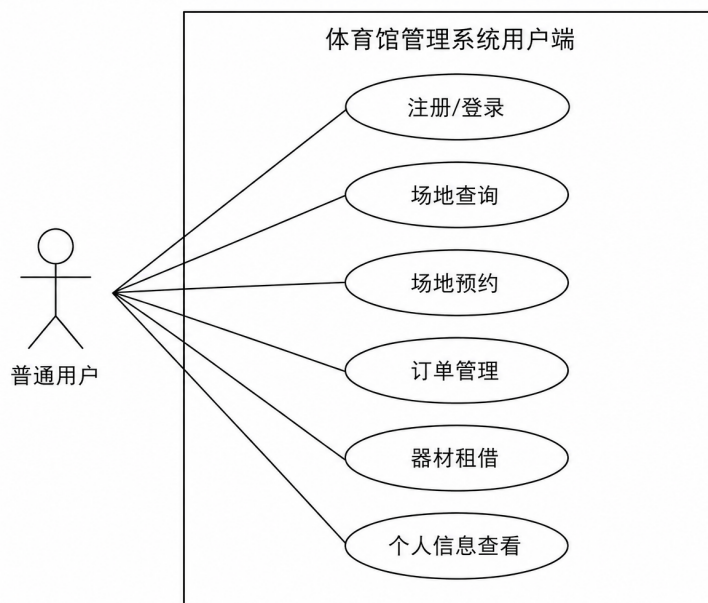


图 3.1 用户端功能用例图

3.2.2 管理端功能需求分析

管理端主要是对体育馆管理员进行基础数据的管理以及业务巡检工作。管理员登录后会显示后台首页的概览，即场地总数、开放场地数、订单总订单量、器材库存以及注册用户数等；管理员可以新增、修改场地信息，也可以设置场地上下架状态；管理员可以在订单管理中查看全部订单，可以按照状态筛选；管理员可以在器材管理中更新器材数量和租借单价，并对未还的设备执行归还；管理员可以查看所有的注册用户，并可以改变用户的权限。

管理端与用户端的区别在于管理员的操作会改变其它用户的可以查看或者使用的数据。场地下架之后，用户端不能继续预订这个场地；器材总数量发生变化时，借出的数量也要随之变动；用户的权限被提升了以后，这个账号就可以进入到后台页面了。所以管理端的需求不能只描述出“可以增删改查”，还要加上操作校验、权限控制等要求。本系统的后端管理接口使用管理员权限来限制访问，前端路由守卫可以降低出现后台页面误入的情况。

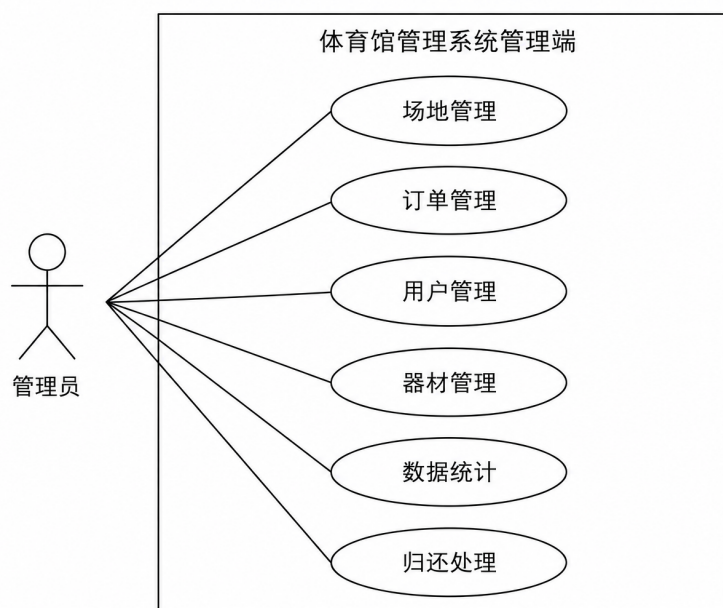


图 3.2 管理端功能用例图

3.3 非功能需求分析

3.3.1 性能与安全需求分析

性能要求主要是预约高峰期。校园体育馆的访问量一般不会像大型商业平台那样大，在周末、傍晚和活动前后，场地查询、时段刷新和下单请求都会集中出现。因此系统需要保证常用列表接口可以分页返回，预约页面只请求当前场地和当前日期的时段数据，避免一次性加载过多无关的内容。订单创建接口要将冲突查询、订单写入一起放到同一个业务流程里来，以降低重复预约发生的概率。

安全需求主要包含身份认证、权限控制、输入校验这三个方面。用户登录成功之后，后端会发出 JWT，前端在之后的请求里使用 Authorization 请求头传递 token，后端拦截器从请求头中提取出用户的编号以及所扮演的角色。普通用户访问个人订单、租借记录时，应该以后端解析出来的用户编号为准，不能相信前端传入的用户编号；管理接口还需要对用户的角色进行判断，是否为 admin。场地时间、订单状态、器材数量等参数也要经过后台的二次确认才能让用户绕过前端页面直接调用接口。目前系统仍处于课程设计版本，密码存储、接口限流等安全措施还存在改善的空间，在后面的展望中也会有所提及。

3.3.2 易用性与可维护性分析

易用性看上去是个软性指标，但是对实践有着非常重要的作用。普通用户对于系统的操作没有耐心，走完一条路绕一圈，错误的提示无法看懂就放弃；管理员容忍度比较高，但是每打开一页需要点击几十次才能完成的系统，在遇到交互异常的时候效率就会受到影响。在系统方面主要的做法就是尽量减少常用的操作次数，表单失败时给出错误

原因的说明，列表页提供搜索、分页等功能。

可维护性更多是针对代码的。前后端分离可以允许前后两端独立更新，修改哪个地方就有机会发现 **bug**，后端用控制器、服务、**mapper** 三个层次来组织代码，定位错误时可以直接从接口路径入手查找；前端按照页面拆成组件，公共逻辑放到 **composables** 中去。该种结构对以后接手开发的同学比较有利。

第 4 章 系统总体设计

4.1 系统架构设计

4.1.1 前后端分离架构设计

该系统是前后端分离结构的。前端项目主要实现页面展示、路由跳转、表单校验以及接口调用，后端项目主要是做登录认证、业务规则的判断以及对数据库的操作。两端之间通过 HTTP 接口传递 JSON 数据，前端统一以 /api 作为接口前缀，并在 Axios 请求拦截器中携带 JWT；后端使用 Spring Boot 提供接口，控制器根据认证、场地、订单、器材、用户等业务进行划分，再由 service 层处理具体逻辑，mapper 层完成数据库访问。

这种架构更适合本课题，系统既存在用户端又存在管理端，但是它们共享同一批后端数据。用户端的场地列表调用 /api/venues 查询可预约场地，管理端通过场地维护接口修改同一张场地表；用户提交订单后，个人订单页和后台订单页都能看到相应的记录，但是查询范围不同。前后端分离之后，页面布局及交互修改不会影响后端业务规则，后端接口也可以在不改变页面结构的情况下补充校验逻辑。

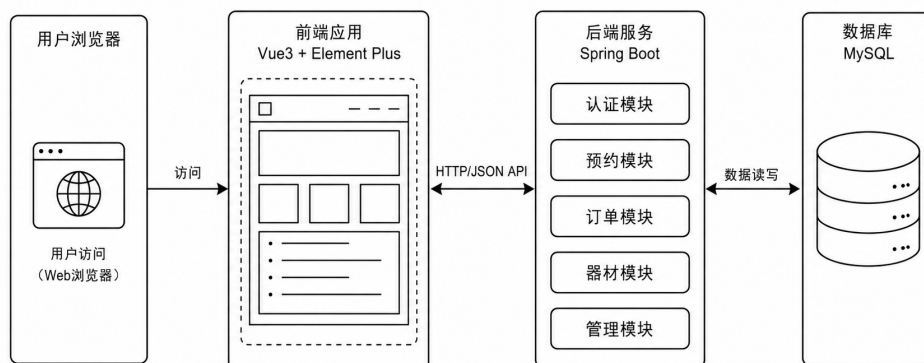


图 4.1 前后端分离架构图

4.1.2 系统模块划分设计

按照使用角色和业务边界，可以将系统分为用户端模块、管理端模块和公共支撑模块。用户端模块面向普通师生，包括注册登录、场地查询、预约下单、我的订单、器材租借、租借记录查询等。该模块重视操作路径短，用户选择场地之后只需要确定日期、开始时间和结束时间，即可生成待支付订单。

管理端模块面向管理员，有后台首页、场地管理、订单管理、器材管理、用户管理。本模块的重点并不是界面的数量，而是维护数据的一致性，场地上下架会影响用户端是否展示，器材数量修改要考虑已经借出的数量，用户角色调整会改变后台的访问权限。

公共支撑模块是第一、二类功能的延伸，包含 JWT 鉴权、管理员权限判断、统一返回结果封装、全局异常处理以及分页查询等。将这些支撑能力从各个业务接口中抽取出来，可以减少各业务接口中的重复代码，也可以使得前端对于成功和失败的响应使用较为一致的处理方式。

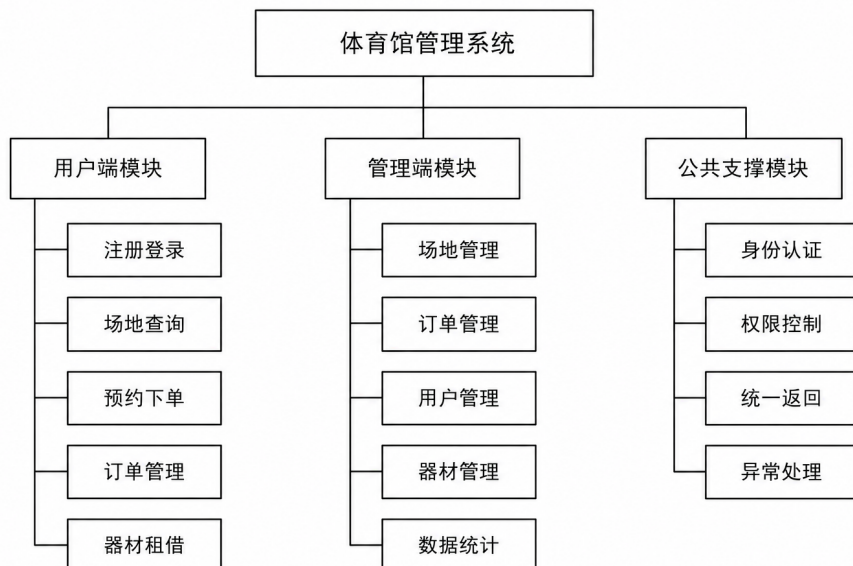


图 4.2 系统功能模块图

4.1.3 核心业务流程设计

系统中两条主线需要单独画出流程。第一条是场地预约：用户登录后进入场地列表，根据题目给出的条件选择一个合适的场地、日期和时段后提交预约申请，系统后端校验冲突并生成订单，订单进入“待支付”状态等用户后续支付或取消。第二条是器材租借：用户在器材列表里查看库存、提交租借申请、后端扣减可借数量并生成租借记录，当管理员在后台进行归还操作的时候再把数量加上。两条主线提前梳理好之后，就可以在后面的设计与关系图中更好地呈现出来。

体育馆管理系统核心业务流程图

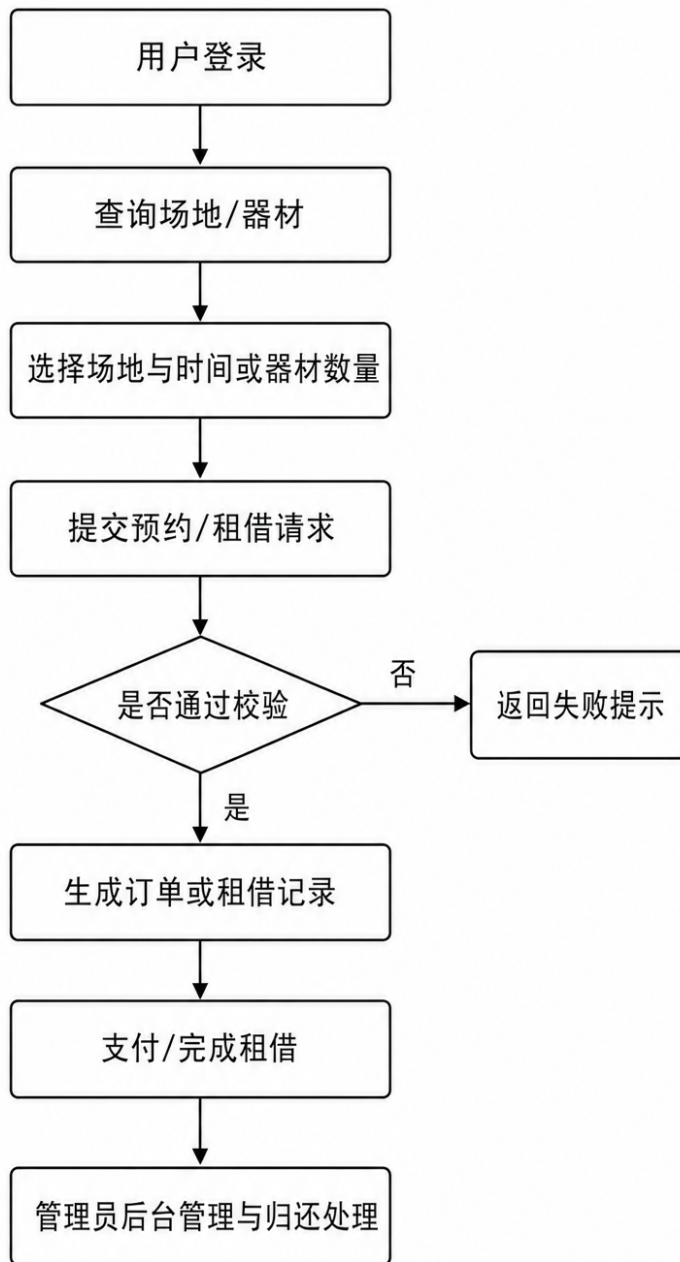


图 4.3 核心业务流程图

4.2 数据库设计

4.2.1 数据库概念结构设计

概念结构这一步主要是从业务里抽象出”东西”。本系统抽出了六个核心实体：角色、用户、场地、订单、器材、租借记录。它们之间的关系并不复杂——一个角色对应多个用户，一个用户可以下多个订单，一个场地也可以承载多个订单；器材和租借记录之间通过租借数量、归还状态来反映库存的变化。这些关系合在一起，就构成了系统的业务语义^[1]。把它们画成 ER 图，能比较直观地看到各个对象的位置和走向，也方便后面落到具体的数据表上。

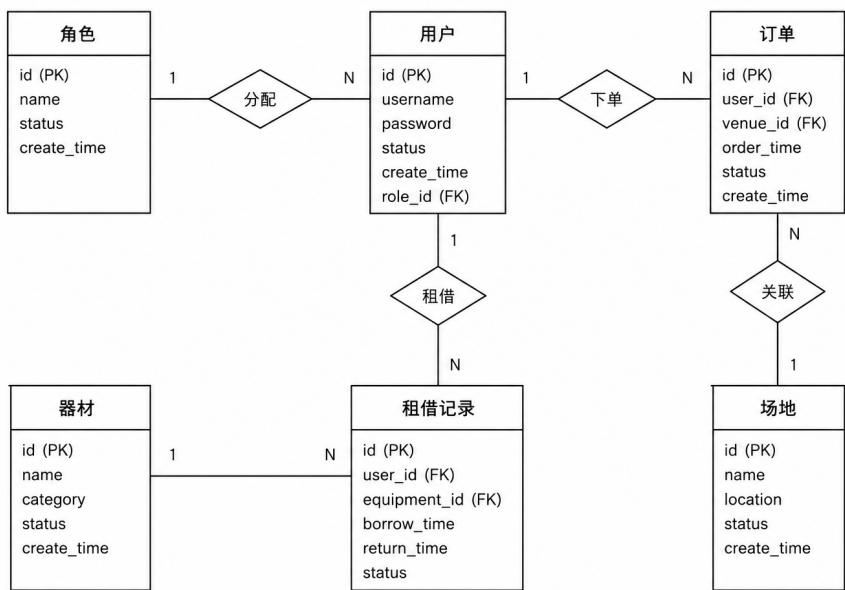


图 4.4 数据库 ER 图

4.2.2 数据库逻辑结构设计

下一步把概念模型落到 MySQL 的具体表上，给每张表定字段、主键、外键和必要的约束。结合项目里的初始化脚本，系统一共有六张业务表：角色表、用户表、场地表、订单表、器材表、租借记录表。它们各自负责一类数据，外键和状态字段把彼此连起来——例如订单表里同时持有 user_id 和 venue_id，租借记录表持有 user_id 和 equipment_id。下面几张表给出每张表的字段细节。

表 4.1 角色表结构设计

字段名	数据类型	约束	字段说明
id	BIGINT	主键, 自增	角色编号
name	VARCHAR(50)	非空, 唯一	角色名称, 取值为 admin 或 user

表 4.2 用户表结构设计

字段名	数据类型	约束	字段说明
id	BIGINT	主键, 自增	用户编号
username	VARCHAR(50)	非空, 唯一	用户名
password	VARCHAR(100)	非空	登录密码
phone	VARCHAR(20)	可空	用户手机号
role_id	BIGINT	非空, 外键	对应角色编号
create_time	DATETIME	非空	用户创建时间

表 4.3 场地表结构设计

字段名	数据类型	约束	字段说明
id	BIGINT	主键, 自增	场地编号
name	VARCHAR(100)	非空	场地名称
type	VARCHAR(50)	非空	场地类型
status	TINYINT	非空, 默认 1	场地状态
open_time	TIME	非空	开放开始时间
close_time	TIME	非空	开放结束时间
price_per_hour	DECIMAL(10,2)	非空	每小时价格
create_time	DATETIME	非空	创建时间

表 4.4 订单表结构设计

字段名	数据类型	约束	字段说明
id	BIGINT	主键, 自增	订单编号
order_no	VARCHAR(32)	非空, 唯一	订单号
user_id	BIGINT	非空, 外键	下单用户编号
venue_id	BIGINT	非空, 外键	预约场地编号
book_date	DATE	非空	预约日期
start_time	TIME	非空	开始时段
end_time	TIME	非空	结束时段
total_amount	DECIMAL(10,2)	非空	订单总金额
status	TINYINT	非空, 默认 0	订单状态
create_time	DATETIME	非空	订单创建时间

表 4.5 器材表结构设计

字段名	数据类型	约束	字段说明
id	BIGINT	主键, 自增	器材编号
name	VARCHAR(100)	非空	器材名称
total_qty	INT	非空	器材总数量
available_qty	INT	非空	当前可借数量
price_per_hour	DECIMAL(10,2)	非空	每小时租借价格
create_time	DATETIME	非空	创建时间

表 4.6 租借记录表结构设计

字段名	数据类型	约束	字段说明
id	BIGINT	主键, 自增	租借记录编号
user_id	BIGINT	非空, 外键	租借用户编号
equipment_id	BIGINT	非空, 外键	器材编号
quantity	INT	非空	租借数量
status	TINYINT	非空, 默认 0	租借状态
create_time	DATETIME	非空	租借时间
return_time	DATETIME	可空	归还时间

第5章 系统详细设计与实现

5.1 场地列表与筛选实现

场地列表页是普通用户进入到系统以后的最主要的一个入口。页面上没有使用复杂的查询条件，仅保留了场地类型的筛选以及分页浏览，因为体育馆场地数量比较少，一般用户都会先根据运动项目来查找，再按照开放时间与价格决定是否预约。前端在 `VenueList.vue` 中调用 `/api/venues` 接口，请求参数包括页码、每页数量和可选的 `type`。接口返回之后，页面会显示场地名称、类型、开放时间、单价以及状态，并且把“预约”按钮放在每个场地条目旁边。

后端查询逻辑在场地相关接口中进行。系统将所有位于上架位置的场地都对普通用户开放预约权限，管理员可于后台查询全部的场地信息，并可对它们的状态进行调节。这样处理之后，场地临时维修或者停止对外开放的时候，不需要删除数据，只需要把状态改为下架即可。数据库中保存着这个场地历史订单记录，之后统计或者追溯的时候不会丢失。

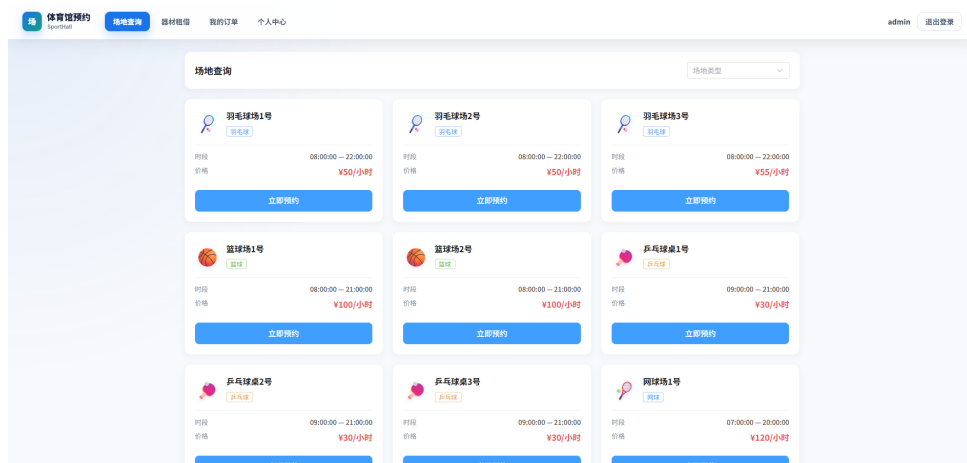


图 5.1 场地列表与筛选页面

5.2 预约时段选择与下单实现

预约页面的主要功能就是“先让用户看到哪些时间不能选”。用户在场地图像列表中到达预约页时，前端会根据路由中的场地编号向后台发送请求场地详情的指令，接收到的开放起止时间和每个小时的单价都会返回。用户选择日期后，页面再请求 `/api/venues/{id}/schedule`，后端返回当天已经被占用的时段。前端根据获取到的数据产生开始时间、结束时间和时间段的下拉框，被占用的整点时段会显示在“时间段概况”当中，结束时间选项会在出现被占用的时段时停止展开，防止用户在页面上选择跨过已经约定的时段。

提交预约的时候，前端向 `/api/orders` 提供场地编号、预约日期、开始时间、结束时

间。后端不会直接接受前端的返回值，会再次核对场地是否在场，是否上架，预约日期是否晚于当前日期，起止时间是否有效，以及是否在场地开放时间内。冲突判断采用区间重叠条件来完成，即已有订单的开始时间早于本次结束时间，而已有订单的结束时间晚于本次开始时间；状态为待支付和已支付的订单都被认为是被占用的。校验合格之后，系统就产生订单号，根据预约时长以及场地单价来算出总价，然后将此价存入待付款订单中。

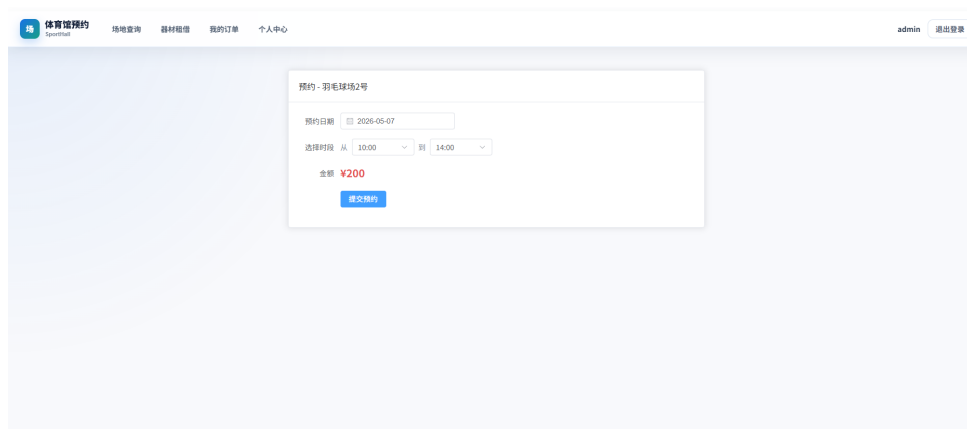


图 5.2 预约时段选择与下单页面

5.3 订单列表与状态操作实现

订单列表页是对用户自己预约记录的展示页面。前端通过 `/api/orders` 获取分页信息，表格上订单号、预约时间、预约时间段、金额、状态都显示出来。系统把订单状态分为待支付、已支付和已取消三类，根据状态来决定是显示还是不显示支付和取消的操作。用户完成预约之后就会进入到个人订单页面，方便用户立即处理刚产生出来的待支付订单。

订单状态操作的限制只在后端。支付接口、取消接口都通过订单编号去查询记录，判断该订单是否是当前登录用户的所有订单，同时判断该订单的状态是否还为待支付。只有待支付订单才能变成已支付或者已取消状态，已经完成或者已经取消的订单不能再进行操作。因此即使前端页面没有立即刷新，后端也会阻止不合理的状态改变。



图 5.3 订单列表与状态操作页面

5.4 器材租借与记录查询实现

器材租借功能比场地预约流程短，但是要对库存一致性有要求。用户端器材页用 /api/equipments 获取器材列表，页面显示器材名称、总数量、可借数量和租借单价。用户输入数量之后提交租借申请，前端会对数量进行判断，数量大于 0 且不超过页面显示的可以借的数量；后端在设备服务中根据最新的库存重新验证，防止用户通过页面绕过或者在库存变化的时候提交已经过期的数据。

租借成功之后，后端会先从器材表里扣除可用量，然后写入一条租借记录，记录的状态为“租借中”。归还操作是由管理员来完成的，接口会根据租借记录编号去查找到原始的记录信息，看是否还在被占用的状态里。只有确认是原来的记录时，才可以对它进行增补操作，将其可借数量增加到原有的数字中来，并且把归还的时间写进去。租借、归还均在事务方法中完成，目的在于避免只对库存做了修改或者只对租借信息进行了更改而造成的中间状态。

租借记录

状态筛选

记录ID	用户ID	器材ID	数量	租借时间	归还时间	状态	操作
19	5	8	1	2025-12-12T15:00:00		租借中	归还
18	8	7	2	2025-12-12T09:00:00		租借中	归还
17	3	2	1	2025-12-11T10:00:00		租借中	归还
16	2	1	2	2025-12-10T14:00:00		租借中	归还
15	7	4	1	2025-12-08T18:00:00	2025-12-08T20:00:00	已归还	
14	6	6	2	2025-12-02T10:00:00	2025-12-02T12:00:00	已归还	
13	5	2	1	2025-11-26T19:00:00	2025-11-26T21:00:00	已归还	
12	4	8	2	2025-11-22T10:00:00	2025-11-22T12:00:00	已归还	
11	3	1	3	2025-11-16T09:00:00	2025-11-16T11:00:00	已归还	
10	10	5	1	2025-11-11T14:00:00	2025-11-11T16:00:00	已归还	
9	2	3	2	2025-11-02T10:00:00	2025-11-02T12:00:00	已归还	
8	9	7	4	2025-10-29T18:00:00	2025-10-29T20:00:00	已归还	
7	8	6	3	2025-10-24T10:00:00	2025-10-24T12:00:00	已归还	
6	7	2	2	2025-10-21T16:00:00	2025-10-21T18:00:00	已归还	
5	6	4	2	2025-10-11T07:00:00	2025-10-11T09:00:00	已归还	
4	5	1	1	2025-10-09T18:00:00	2025-10-09T20:00:00	已归还	
3	4	3	2	2025-09-06T09:00:00	2025-09-06T11:00:00	已归还	
2	3	2	1	2025-09-03T18:00:00	2025-09-03T20:10:00	已归还	

图 5.4 器材租借与记录查询页面

5.5 场地管理页面实现

场地管理页给管理员提供对场地基本资料的维护。页面主要是由表格和弹窗表单组成的，表格中显示场地名称、类型、开放时间、单价、状态；新增和修改都使用同一个表单，在编辑时会把当前行的数据带入到表单当中。上下架的操作单独设置为状态开关，在状态开关的上部可以自由调节场地开放状态，不需要打开编辑弹窗就可完成上下架操作。

后台对场地维护接口进行了管理员角色判断，普通用户即使知道接口地址也不能新增或者修改场地。新增或者修改的时候，系统会对名称、类型、开放时间、价格等进行校验，开放开始时间必须早于结束时间，价格不得小于 0。由于用户端预约页的生成可选时段会用到场地开放时间，因此这些校验直接会影响到后面预约流程能否进行。

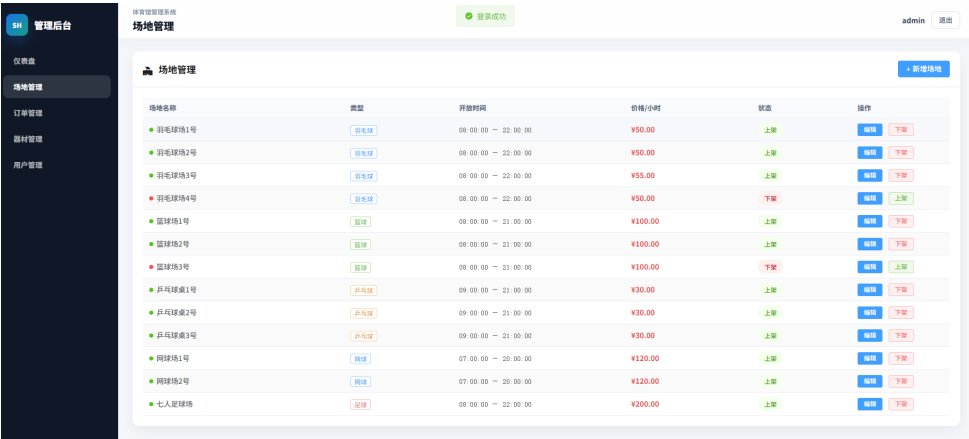


图 5.5 场地管理页面

5.6 订单与用户管理实现

后台订单管理页是管理员查看系统内所有的预约记录的界面。相比用户订单页来说，后台查询没有被限定当前用户编号的范围，按照订单状态和分页条件返回数据。该页面主要是实现对于已经支付的订单进行查询、对于已经取消的订单进行查询，以及对于近期预约的查询功能。由于本系统目前还没有接入真实的支付平台，支付操作仍然通过系统内部的状态变化来模拟，所以后台订单管理更多的是起到查询、核对的作用。

用户管理页是对注册的用户进行查看和角色调整。系统中的角色分为普通用户、管理员两种，前端路由会根据本地保存的角色信息来控制后台入口，但是真正的权限边界还在后台接口上。管理接口调用之前会先判断 JWT 中是否有 admin 这个角色，没有就拒绝处理。该种方式可以避免只依靠前端路由产生越权访问情况。

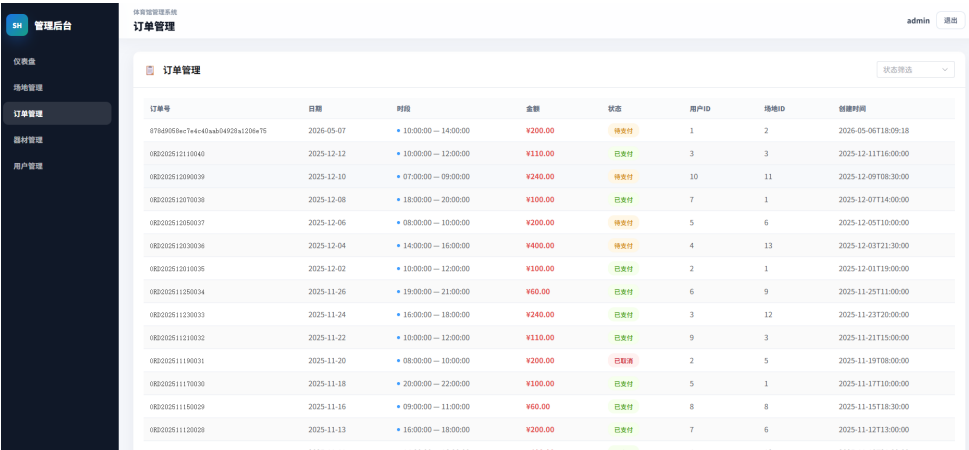


图 5.6 订单管理页面

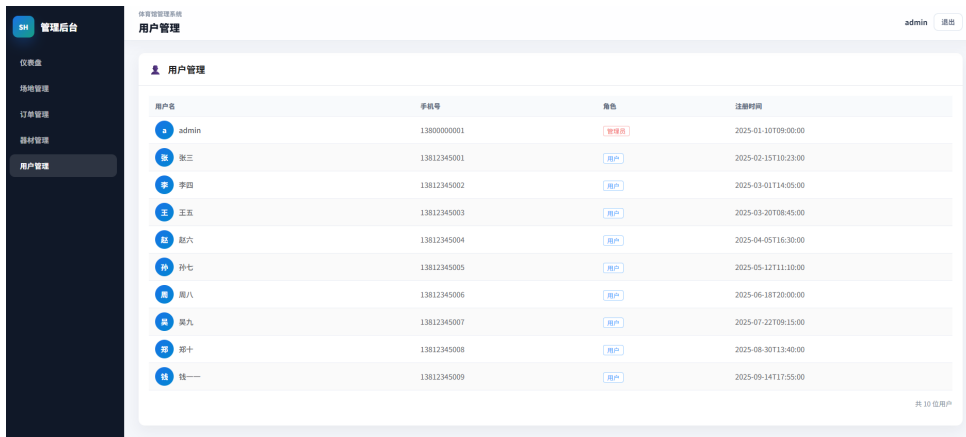


图 5.7 用户管理页面

5.7 器材信息维护实现

器材信息维护页用来管理器材名称、总数量、可借数量、租借单价。新增器材时，如果管理员没有单独填写可借数量，系统默认可借数量为总数量；修改器材时，系统不能简单地覆盖库存，因为已经存在的尚未归还的租借记录会受到影响。后台先求出已有借出的数量，再判断新的总数小于已借出的多少个。新总的大小太小了，接口就会拒绝更新，不能出现可以借的量为负数的情况。

这部分逻辑放到后端实现，主要是因为前端页面只能体现打开页面时的数据状态，库存会随着用户租借或者管理员归还发生变化。后端更新时重新读取数据库中旧的记录，可以减少库存计算使用过期页面数据的问题。

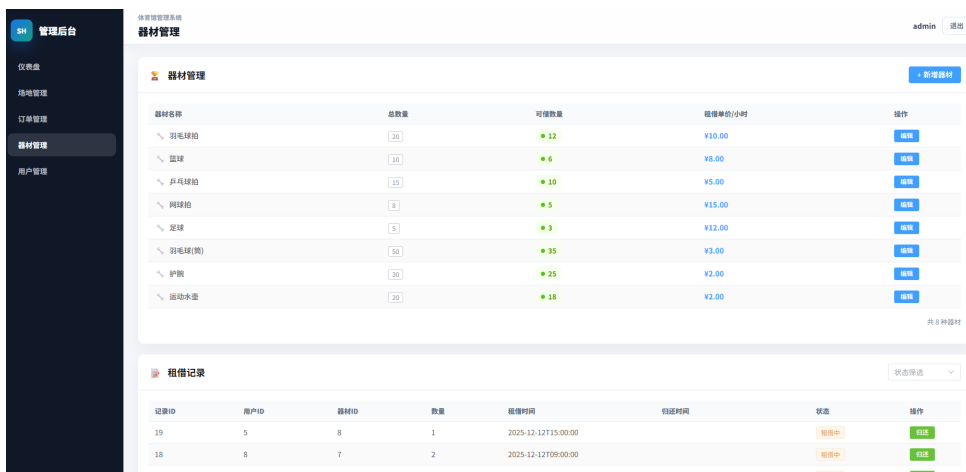


图 5.8 器材信息维护页面

5.8 数据统计看板实现

后台首页的数据看板是给管理员提供的一个快捷的入口。页面加载的时候同时请求场地、订单、器材、用户相关接口，在前端计算出开放场地数、订单总量、已支付订单

数、器材总库存、可借库存、注册用户数量等指标。页面会以近 7 日为单位，用简单的柱状图展示最近的订单情况以及最近的预约日期。由于本系统数据量小，可以全部在前端进行统计，代码也较易理解。

需要指出的是，目前的看板还只是基础统计，并没有单独设计出统计表或者复杂的图表接口。当后期订单数量增多时，前端仍然从后端拉取过多的订单再进行计算是不可取的，应该改为后端提供聚合接口，例如按日期统计订单数量、按场地类型统计使用率等。本课题现阶段保留简单的实现是为了满足管理员查看系统运行情况的需求。

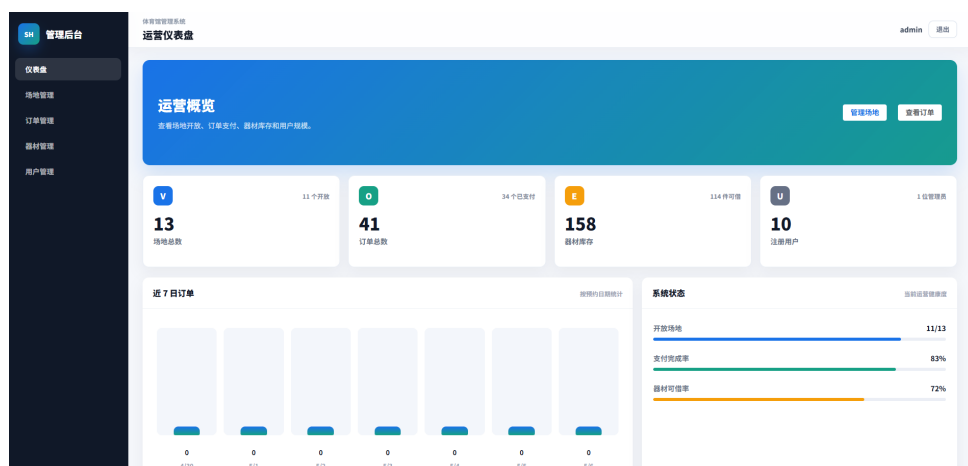


图 5.9 数据统计看板页面

第 6 章 系统测试

6.1 测试环境与方法

为了检验体育馆管理系统是否满足设计要求，本文在前后端联调之后对系统进行综合测试。测试过程主要是对用户端预约业务、器材租借业务、管理端后台维护功能进行测试，测试内容为功能正确性、业务流程完整性、异常处理的有效性、系统运行的稳定性等各方面。根据本课题的实现情况，本文主要用黑盒测试的方法对系统的输入和输出进行对比，来检验系统是否达到了预期的设计目的^[12]。

本次测试所用的环境有前端运行环境、后端服务环境、数据库环境以及浏览器环境这四种。测试时用本地部署的方式运行前后端项目，前端主要实现页面访问和交互，后端主要是处理接口请求以及进行业务校验，数据库用来保存用户、场地、订单、器材等重要的业务数据。具体的测试环境如表 6.1 所示。

表 6.1 系统测试环境

测试项目	环境配置	说明
操作系统	Windows 11	系统开发与测试使用环境
前端运行环境	Node.js 18, Vue3, Vite	用于启动用户端与管理端页面
后端运行环境	Java 17, Spring Boot 2.7	用于提供业务接口与权限控制
数据库环境	MySQL 8.0	用于存储系统业务数据
开发工具	IntelliJ IDEA, VS Code	用于前后端代码开发与调试
测试浏览器	Google Chrome	用于页面访问与交互测试

6.2 功能测试分析

根据系统业务流程和功能模块划分，本文选取了用户登录、场地查询、预约下单、订单状态操作、器材租借以及后台管理等关键功能作为测试对象。测试时分别输入正常数据和异常数据，观察系统是否能够返回正确结果，并检查各业务环节之间的数据是否保持一致。部分核心功能测试结果如表 6.2 所示。

从测试结果来看，系统各项核心功能均能够按照预期完成。普通用户可以顺利进行场地浏览、预约下单、订单支付与取消以及器材租借等操作，管理员也能够完成场地维护、订单查看、用户管理和器材信息维护等后台操作。在异常场景下，例如未登录访问受限资源、预约时间冲突、器材库存不足和权限不匹配等情况，系统均能够返回明确提示信息并阻止非法操作，说明系统在业务逻辑控制和异常处理方面具备较好的正确性。

综合分析可知，本系统已经基本实现体育馆管理的主要功能目标，能够满足校园场馆日常预约与管理需求。虽然在大规模并发访问、更加复杂的数据统计和更精细的安全控制方面仍有进一步优化空间，但从当前测试结果来看，系统在功能完整性、流程连贯性和运行稳定性方面均达到了本科毕业设计的预期要求。

表 6.2 核心功能测试结果

测试编号	测试内容	预期结果	测试结果
T1	输入正确用户名和密码进行登录	登录成功并进入系统首页	通过
T2	按场地类型筛选场地列表	返回符合条件的场地数据	通过
T3	选择未被占用时段提交预约	成功生成待支付订单	通过
T4	选择已被占用时段提交预约	系统提示预约冲突并拒绝创建订单	通过
T5	对待支付订单执行取消操作	订单状态更新为已取消	通过
T6	提交器材租借请求且库存充足	生成租借记录并减少可借数量	通过
T7	提交器材租借请求且库存不足	系统提示库存不足并拒绝租借	通过
T8	管理员修改场地状态和器材信息	数据更新成功并在页面中正确显示	通过

第 7 章 总结与展望

7.1 研究总结

本文用 Java Web 技术实现了一个体育馆管理系统。本系统以校园体育馆里较为常见的场地预约、器材租赁为依托，分别对普通用户、管理员两大类用户的使用需求进行分析，最后完成了前后端分离的架构设计、数据库表设计、主要功能实现及测试工作。本文在整体开发的过程中并没有将体育馆管理抽象为一个巨大的平台，而是选取了预约、订单、器材、后台维护等几个具体环节来开展工作，保证系统可以形成可以演示和测试的闭环。

用户端的功能有注册登录、场地列表、预约下单、我的订单、器材租借和个人记录查询等。预约流程中，系统会根据场地开放时间 & 当天已占用时段生成可选时间段，提交订单前后端都会进行校验；订单生成之后可以进行支付或者取消，状态改变会反映到个人订单列表上。管理端实现了场地管理、订单查看、器材维护、用户角色管理以及数据看板等主要功能，管理员能够对基础数据进行维护，也可以在后台首页上看到目前系统的大概运行状况。

在技术实现方面，前端采用 Vue3、Vite 和 Element Plus，后端采用 Spring Boot、MyBatis-Plus 和 MySQL。系统用 JWT 保存登录后的身份信息，在后端接口中按照用户编号和角色来限制访问范围；统一返回对象、全局异常处理使前端可以以类似的方式去处理接口的结果。测试阶段主要是针对登录认证、场地预约、订单状态流转、库存扣减、归还处理和管理员权限等流程进行测试，结果证明本系统可以完成毕业设计所要达到的功能要求。

但是系统存在一定的不足。由于课题时长受限以及开发范围的制约，目前支付功能仍然使用状态模拟的方式，并未与真实的支付通道对接；密码存储、日志审计、接口限流等安全保障措施不齐全，预约冲突依靠业务查询和事务处理解决，在更高的并发下仍需加强数据库约束或者使用锁。这些不足可以明确系统边界的把握情况，并给后面改进建设给予线索。

7.2 后续展望

之后可以从业务完整性上进行完善。真实使用时预约支付不应该只是订单状态的切换，还可以接入支付接口或者校园卡系统；预约成功、订单取消、器材归还超时等节点也可以增加短信、邮件或者站内通知提醒。普通用户一般是通过手机进行预订，因此前段可以更加深入地改善小屏幕上表格所占用的部分，在减少用户阅读和使用时间的基础上进一步提高效率。

安全性也是后面需要加强的部分。目前系统适合课程演示，但是要接近真实部署的话，密码应该用加盐哈希存储，登录失败次数也应该有限制，关键管理操作也应当记录日志，管理员权限也可以从简单的 admin/user 二分扩展为更细的权限项。将场地维护、订单查看、用户角色管理拆分，避免使用同一个管理员账号具有太大权限。

就性能以及数据分析而言，系统能渐渐地把统计逻辑从前端迁移至后端聚合接口之中。随着订单数量的增加，后台首页不能再一次性读取大量的订单然后交给前端去计算，而应该由后端按日期、场地类型或者状态直接返回统计结果。还可以添加场地使用率、热门时间段、器材借还情况等分析项，利于管理员对场地进行调度安排。

最后，学校如果有统一的身份认证、校园卡或者门禁系统等，那么本系统就可以和这些平台进行对接。因此用户不必另设账号，管理员亦可将预约、支付、入馆核验等关联起来。由于本科毕业设计开发条件所限，本文并没有实现这些扩展功能，但是从系统的模块划分来看，在认证、支付、统计模块上继续进行迭代是可行的。

参考文献

- [1] Feng J. Retraction note: Designing an artificial intelligence-based sport management system using big data[J/OL]. Soft Computing, 2026:1-1. DOI: 10.1007/S00500-026-11275-1.
- [2] Ding J, Su Y. Retraction note: A teaching management system for physical education in colleges and universities using the theory of multiple intelligences and svm[J/OL]. Soft Computing, 2026:1-1. DOI: 10.1007/S00500-025-11129-2.
- [3] 张胜涛. 计算机技术在体育馆器材管理中应用探究[J]. 文体用品与科技, 2022(19):178-180.
- [4] 林悠阳. IT 服务标准在 GT 体育馆运维服务的应用研究[D/OL]. 厦门大学, 2022. DOI: 10.27424/d.cnki.gxmd.2022.004491.
- [5] 罗琼书. 计算机技术在体育馆器材管理中的运用[J/OL]. 科技创新导报, 2021, 18(11):174-176. DOI: 10.16660/j.cnki.1674-098X.2103-5640-1458.
- [6] 王古将. 高校体育馆管理软件系统研究与设计[D/OL]. 东华大学, 2020. DOI: 10.27012/d.cnki.gdhuu.2020.000446.
- [7] 张焱, 童莲. 高校体育馆预约管理系统的设计与实现[J]. 花炮科技与市场, 2019(04):214.
- [8] 罗超. 基于 java 的健身房管理系统关键技术研究[J/OL]. 信息记录材料, 2019, 20(08):159-160. DOI: 10.16009/j.cnki.cn13-1295/tq.2019.08.105.
- [9] 吴涛. 学校体育馆智能灯光管理系统[J/OL]. 电子世界, 2018(11):45-46. DOI: 10.19353/j.cnki.dzsj.2018.11.019.
- [10] 张衡, 任崇婷. 基于 WEB 的游泳馆会员管理系统的分析与设计[J]. 中国新通信, 2017, 19(08):48.
- [11] 万世友. 高校体育馆网络管理信息系统的设计与实现[D]. 吉林大学, 2016.
- [12] Zhao Z, Wang B, He X, et al. Can artificial intelligence enhance the stability of supply chain systems for sports enterprises? insights from systems theory and supply chain management theory[J/OL]. Systems, 2026, 14(3):299-299. DOI: 10.3390/SYSTEMS14030299.

致谢

毕业设计和论文写到这里，也算给大学阶段的学习画上了一个阶段性的句号。

首先要感谢我的指导教师。在选题、系统实现和论文修改过程中，老师都给了我很多具体的意见。尤其是在我思路不清、进度比较慢的时候，老师的提醒和修改建议对我帮助很大，让我能够一步一步把内容补完整、把问题改明白。

同时也感谢学院各位老师在大学期间的教学和指导。课程学习中接触到的数据库、Java、软件工程和前后端开发相关知识，都是这次完成课题的基础。没有平时课程里的积累，这篇论文和这个系统都很难顺利完成。

还要感谢身边同学在毕业设计期间的交流和帮助。无论是讨论功能实现，还是排查一些开发和排版上的小问题，这些交流都让我少走了不少弯路。

最后感谢家人一直以来的支持和理解。毕业设计这段时间里，正是因为有他们的包容和鼓励，我才能比较安心地完成系统开发和论文撰写。

在此，向所有给予我帮助的人表示感谢。